Universiteit Antwerpen
Faculteit Wetenschappen

Project Software Engineering

Project 24-25: Traffic simulator

What you will learn

- **Quality control: <u>tests, contracts</u>**
- **Collaboration in group (e.g., version control with GitHub)**
- **Object-oriented design**
  - Separation of concerns: split into multiple files and classes
  - Managing dependencies: Includes, contructor parameters, …
  - Decoupling
  - **=> Make your code flexible and adaptable**
- **Customer interaction:**
  - Interpreting a specification
  - Negotiating

Traffic simulator

Software for traffic simulator

- **Vehicles**
- **Roads**
- **Movement**
- **Functionality described in use-cases**
  - Now: specification 1.0
  - Later: specification 2.0

Use-cases for specification 1.0

Use-Case	Prioriteit
1: Invoer	
1.1. Verkeerssituatie inlezen	VERPLICHT
1.2. Voertuiggenerator inlezen	BELANGRIJK
2: Uitvoer	
2.1. Simpele uitvoer	VERPLICHT
3: Simulatie	
3.1. Rijden van voertuig	VERPLICHT
3.2. Simulatie van verkeerslicht	VERPLICHT
3.3. Automatische simulatie	VERPLICHT
3.4. Simulatie met voertuiggenerator	BELANGRIJK

General requirements

Tools

- C++17
- Buildsystem (CMake, CLion IDE)
- Tests (Google Test)
- Contracts (DesignByContract.h)
- XML input files (TinyXML)
- Documentation (Doxygen)
- Collaboration (GitHub)

Evaluation criteria

- Example evaluation form (Blackboard)
- Non-functional requirements (Blackboard)
- Quality control: automatic tests, contracts
- Code quality: OO design, code conventions (Blackboard)
- Good project management: Documentation, planning

Example evaluation form

- Pay close attention to this!!

General

Collaboration: ❑ kept working together ❑ work equally divided ❑ both students understand project
Mastery of tooling: ❑ gtest ❑ TinyXML ❑ CMake ❑ CLion ❑ Doxygen ❑ GitHub
Comment:

Tests

❍ Not present (= no tests)	❍ Wrong usage (= manual test)	❍ Limited (=not everything is tested) - domain tests	❍ Sufficient (= input) - input tests - multiple errors in input	❍ Good (= output) - output tests - scenarios	❍ Excellent (= test code good) - modular - code duplication

Comment:

Contracts

❍ Not present (= no asserts)	❍ Wrong usage (= no pre-post) - not in .h file - private/protected	❍ Limited (=minimum) - properlyInit	❍ Sufficient (= only pre) - getters	❍ Good (= sometimes pre- post) - setters - output	❍ Excellent (= sometimes pre- post) - domain model movements

Comment:

Planning

❍ Not present (= no planning submitted)	❍ Wrong usage (= to late)	❍ Limited (=not everything)	❍ Sufficient (= only required)	❍ Good (= more) - non-required use-cases - as proposed	❍ Excellent (= almost everything) - graph. engine - graphics engine

Comment:

Object-oriented design and programming skills

❑ Basics not present (= not possible to grade)	❑ Basis - code compiles - 0, 1, infinity principle - efficient use of data struc. - code split across files - usage of pointers	❑ Encapsulation - private variables + getters and setters - limited no of functie/ctor parameters - internal data struc. are hidden
❑ Domain model - classes model the domain - class names = nouns - Partially present: 1-on-1 match between classes and responsibilities	❑ ADT - functions = actions taken by object - function names = verbs - functionality close to data (cohesion) - correct usage of inheritance	❑ Modularity (= separation of concerns) - loose coupling - dependency injection - input/output is clearly separated (strings are created in separate output classes) + also in test code

Comment:

Planning

Planning

- **Specification 1.0**
  - Keep track of time (“tijdsblad”)
  - Demonstration of system 1.0 during evaluation 1.0
- **Specificatie 2.0**
  - Promise use-cases in plan 2.0
  - Demonstration of system 2.0 during evaluation 2.0
  - Extra time? => Extra functionalities in plan 2.1
  - Demonstration of system 2.1 during FINAL evaluation 2.1

Planning

- **Keep an eye on updates via Blackboard/e-mail!**

Date	Deadline
Monday, March 17	System 1.0
Thursday, March 27	Plan 2.0
Monday, May 5	System 2.0
Thursday, May 15	Plan 2.1
???	Exam: System 2.1